

COMP90041

Programming and Software Development

Lecture 5 - Arrays

Danny Xu

The University of Melbourne acknowledges the Traditional Owners of the unceded land on which we work, learn and live: the Wurundjeri Woi-wurrung and Bunurong peoples (Burnley, Fishermans Bend, Parkville, Southbank and Werribee campuses), the Yorta Yorta Nation (Dookie and Shepparton campuses), and the Dja Dja Wurrung people (Creswick campus).

The University also acknowledges and is grateful to the Traditional Owners, Elders and Knowledge Holders of all Indigenous nations and clans who have been instrumental in our reconciliation journey.

We recognise the unique place held by Aboriginal and Torres Strait Islander peoples as the original owners and custodians of the lands and waterways across the Australian continent, with histories of continuous connection dating back more than 60,000 years. We also acknowledge their enduring cultural practices of caring for Country.

We pay respect to Elders past, present and future, and acknowledge the importance of Indigenous knowledge in the Academy. As a community of researchers, teachers, professional staff and students we are privileged to work and learn every day with Indigenous colleagues and partners.

WARNING

This material has been reproduced and communicated to you by or on behalf of the University of Melbourne in accordance with section 113P of the *Copyright Act 1968 (Act)*.

The material in this communication may be subject to copyright under the Act.

Any further reproduction or communication of this material by you may be the subject of copyright protection under the Act.

Do not remove this notice

Topics:

- Enumerations
- Wrapper classes
- Arrays





THE UNIVERSITY OF
MELBOURNE

Enumerations

Enumerated Types



- Introduced in Java 5.0
- An enum (short for enumeration) is a **special kind of class** that defines a **fixed set of named constant** values
- Syntax:

```
enum TypeName {VALUE_1, VALUE_2, ..., VALUE_N};
```

- Each constant has a **hidden ordinal** — its position in the list, starting from **0**.
- By **convention** Enum values are named in UPPERCASE_LETTERS, with underlines between them.

Declaring and Using Enums



- Example

```
enum WorkDay {MONDAY, TUESDAY, WEDNESDAY, THURSDAY, FRIDAY};
```

- Declare variables of `enum` Type:

```
WorkDay meetingDay, availableDay;
```

- The variable of `enum` type can be set to one of the values of the `enum` type, or to `null`:

```
meetingDay = WorkDay.THURSDAY;  
availableDay = null;
```

Benefits of Enums vs Strings



- **Not Strings** – but can behave similarly
- **Type-safe** – only allows valid values
- **Efficient** – stored as integers
- Enumerated types can be **compared** using **==** or **equals()**

```
if (meetingDay == WorkDay.THURSDAY)
    System.out.println("Long weekend!");
```

```
WorkDay today = WorkDay.MONDAY;
if (today.equals(WorkDay.MONDAY)) {
    System.out.println("It's Monday!");
}
```


Enums in switch statements



- Enumerated types can be used to control a **switch** statement.
- The switch control expression uses a variable of an enumerated type.
- Case labels do **not** use enum type name, only values

```
enum Flavour { VANILLA, CHOCOLATE, STRAWBERRY }  
switch (favourite) {  
    case VANILLA:  
        System.out.println("Vanilla!");  
        break;  
    case CHOCOLATE:  
        System.out.println("Chocolate!");  
        break;  
}
```

Built-in Enum Methods



Method	Description
protected Enum(String name, int ordinal)	Constructor (used implicitly; you don't call it yourself)
boolean equals(Object other)	Returns true if the values are the same
String name()	Returns the exact name as declared (cannot be overridden)
String toString()	Returns a string representation (can be overridden)
int ordinal()	Returns the constant's position (starting from 0)
int compareTo(EnumType other)	Compares enum positions; returns >0, <0, or 0
static EnumType[] values()	Returns an array of all enum constants
static EnumType valueOf(String name)	Converts a string to the corresponding enum constant

- Note: **toString()** is called automatically when printing or concatenating an enum value.

Enums vs Constants



- Java provides both final constants and enum types — but enums offer more structure, safety, and flexibility. This table highlights the key differences:

Feature	Constants (final)	Enums
Type Safety	No	Yes
Behaviours	No methods or fields	Can have methods and fields
Grouping	Not naturally grouped	Defined as a single type
Iteration	Manual	Built-in values() method
Supports methods/fields	No	Yes (can add attributes/logic)
Use Case	Simple unrelated values	Related, structured values



THE UNIVERSITY OF
MELBOURNE

Live Coding – Part 1 (enumeration)

Live Coding Instruction



- In Subject.java,
 - Create SubjectName enumeration with some subject names.
 - Create a class Subject with a String name and a method to set the subject name.
- In Program.java,
 - Print all subject names in the SubjectName enumeration.
Use some in-built functions of enumeration, such as values() for loop.
 - Create a Subject instance and set a subject name.

See detailed instructions in Ed Workspace/week5.



THE UNIVERSITY OF
MELBOURNE

Wrapper Classes

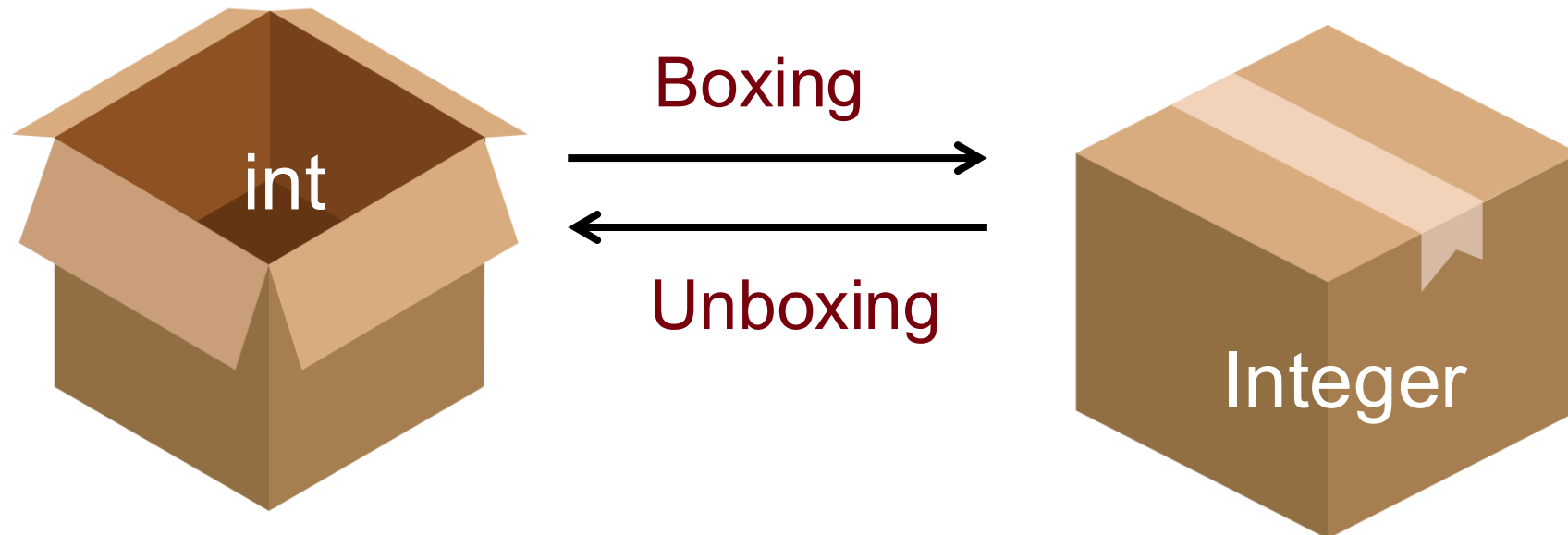
Wrapper Classes



- Primitive variables (int, char, etc.) are **not** Java objects.
- **Wrapper** classes **enable** primitives to behave as objects.
 - boolean -> Boolean
 - byte -> Byte
 - short -> Short
 - long -> Long
 - float -> Float
 - double -> Double
 - char -> Character
 - int -> Integer
- Each has a capital letter, following the Java convention for naming classes, except char and int.

Why Wrapper Classes?

- Wrapper classes contain **useful methods**.
- Example: `Integer.decode(String s)` converts string to Integer.
- **Lists and collections** require Object types, not primitives.



Boxing



- Boxing is the process of **converting** a value of a primitive type to an object of its wrapper class
- Unlike most other classes, a wrapper class does not have a no-argument constructor

```
Integer integerObject = new Integer(42);
```

Unboxing



- Unboxing: **converting** a wrapper class object back to its primitive type.
- The methods for this are:
 - Boolean.booleanValue()
 - Byte.byteValue()
 - Short.shortValue()
 - Integer.intValue()
 - Float.floatValue()
 - Double.doubleValue()
 - Character.charValue()
- Example:

```
int i = integerObject.intValue();
```

Automatic Boxing and Unboxing



- Since Java 5.0, boxing/unboxing can be **automatic**.
- Cleaner code, easier to read.
- Example:

- Instead of this:

```
Integer integerObject = new Integer(42);  
int i = integerObject.intValue();
```

- Automatic:

```
Integer integerObject = 42;  
int i = integerObject;
```

Wrapper Classes - Static Methods



- Wrapper classes have static methods that **convert a correctly formed string representation of a number** to the number of a given type
 - Integer.parseInt ()
 - Long.parseLong ()
 - Float.parseFloat ()
 - Double.parseDouble ()
- **Example:**

```
String numString = "100";  
int number = Integer.parseInt(numString); // Converts to int 100
```

Wrapper Classes - Static Methods



- Wrapper classes also have static methods that **convert from a numeric value to a string** representation of the value
 - Integer.toString(int value)
 - Long.toString(long value)
 - Float.toString(float value)
 - Double.toString(double value)
- **Example:**

```
double numericValue = 123.99;  
String stringValue = Double.toString(numericValue); // returns "123.99"
```

Wrapper Classes - Static Methods



- The **Character** class contains static methods that are useful for string processing.
- To replace lower case characters by upper case equivalents, or vice versa
 - `toUpperCase(char argument)`
 - `toLowerCase(char argument)`
- To return true if the argument is an upper-case letter, and false otherwise, or vice versa.
 - `isUpperCase(char argument)`
 - `isLowerCase(char argument)`
- Example:

```
char lowerChar = 'a';  
char upperChar = Character.toUpperCase(lowerChar); // returns A
```

Wrapper Classes - Static Methods



- The following **Character** class static methods return true if argument is...
 - `isWhitespace(char argument)` // Whitespace (space, tab `\t`, new line `\n`)
 - `isLetter(char argument)` // A letter a-z, A-Z
 - `isDigit(char argument)`
 - `isLetterOrDigit(char argument)`

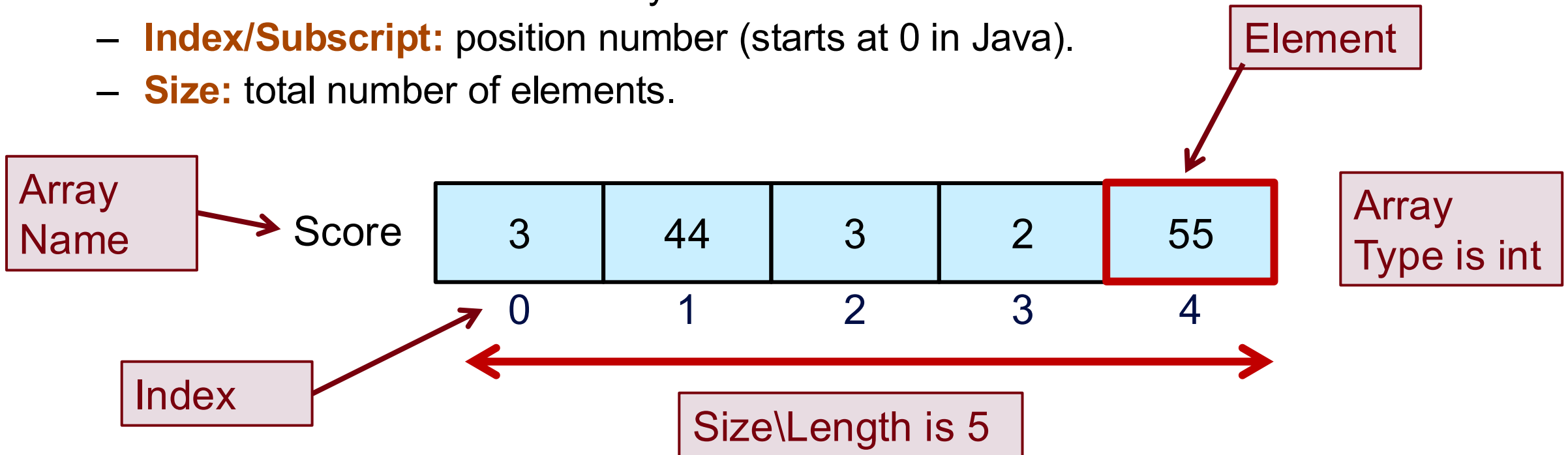


THE UNIVERSITY OF
MELBOURNE

Arrays

What is an Array

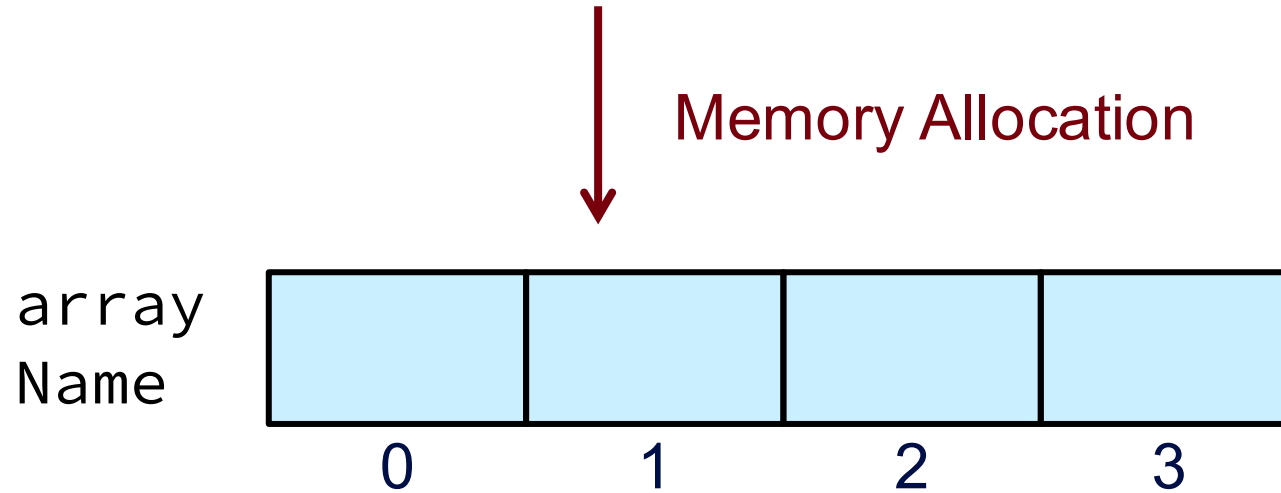
- Array: A data structure used to store and process a **collection of same-type** data.
- Think of it as a list of boxes with numbers on them.
- Key Terms
 - **Element**: one item in the array.
 - **Index/Subscript**: position number (starts at 0 in Java).
 - **Size**: total number of elements.



Declaring and Creating Arrays

- Arrays **Declaration**:

```
type[] arrayName = new type[4];
```



- Arrays can **store any type**:
 - Primitive: `char[]`, `double[]`
 - Objects: `Car[]`

Declaring and Creating Arrays



- Example of one-statement **declaration** that creates **5 elements**: score[0] to score[4]

```
double[] score = new double[5];
```

or

```
double[] score = {1.3, 5.2, 5, 2.1, 5.0}
```

- Example of two-statement version:

```
double[] score;           // Declare the array
score = new double [5];    // Creates the array.
```

Array Size in Java



- Array **Size** (Length):
 - **Number** of **elements** an array can hold.
 - Specified during creation: `new double[5];`
- The size may be **given as an expression** that evaluates to a **nonnegative** integer

```
System.out.print("Enter the size of the array: ");  
int size = scanner.nextInt(); // User input for size  
int[] numbers = new int [size * 2];
```

- Array size can be **determined dynamically** (at runtime).

```
int rand = (int)(Math.random() * 11);  
double[] score = new double[rand];
```

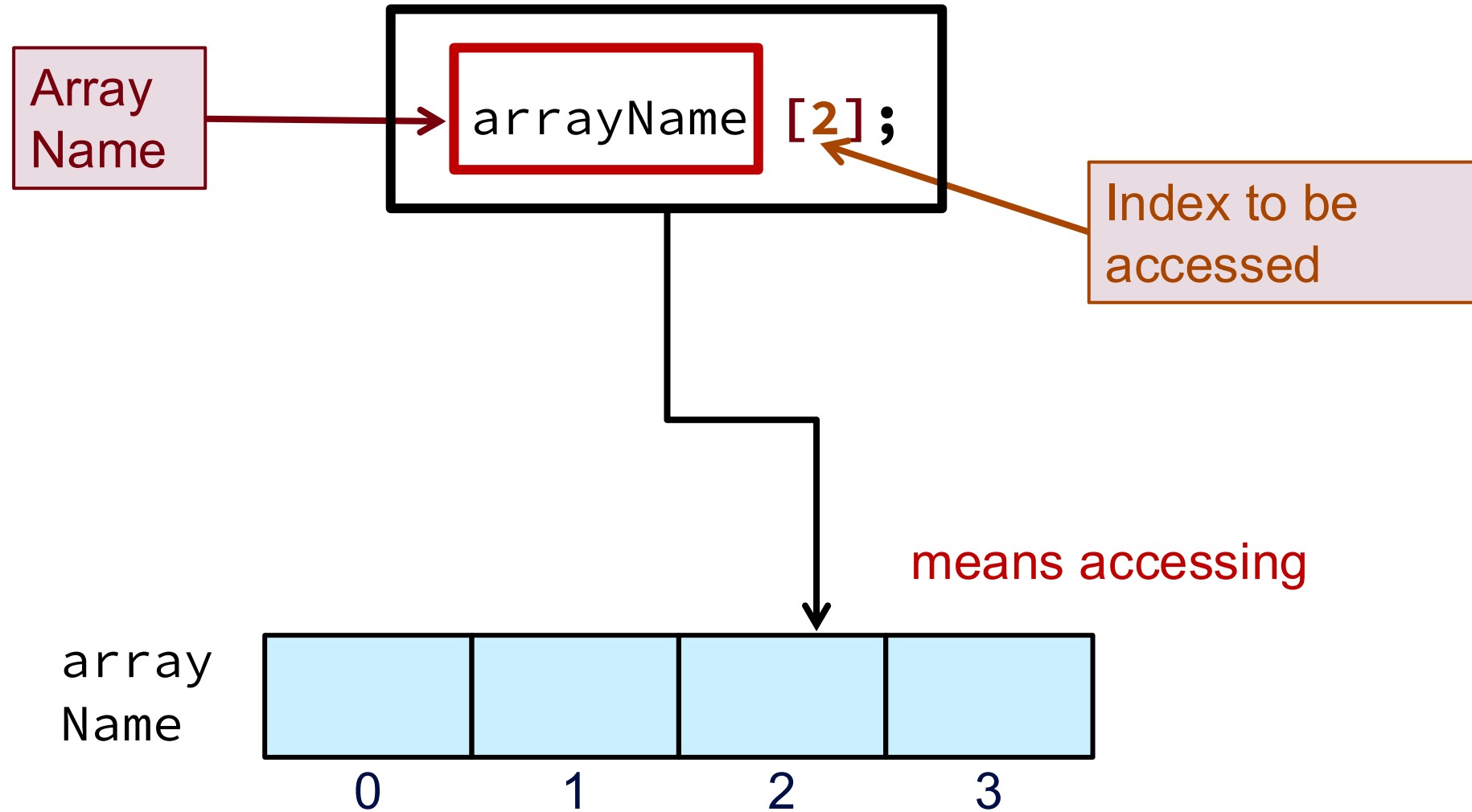
Array Size in Java



- In Java, arrays are **objects**.
- Like other objects, **arrays** have **instance** variables.
- **Every** array has one instance variable called **length**.
 - It is **set automatically** when the array is created.
 - It represents **the size** of the array.
 - It cannot be changed directly (though you can create a new array).
- In Arrays, **length is a variable**, not a method:
 - Arrays: `arrayName.length`
 - Strings: `str.length()`
 - Collections: `list.size()`

```
int size = scanner.nextInt(); // User input for size
int[] numbers = new int [size * 2];
System.out.println("The size of the array: "+ numbers.length);
```

Accessing Arrays

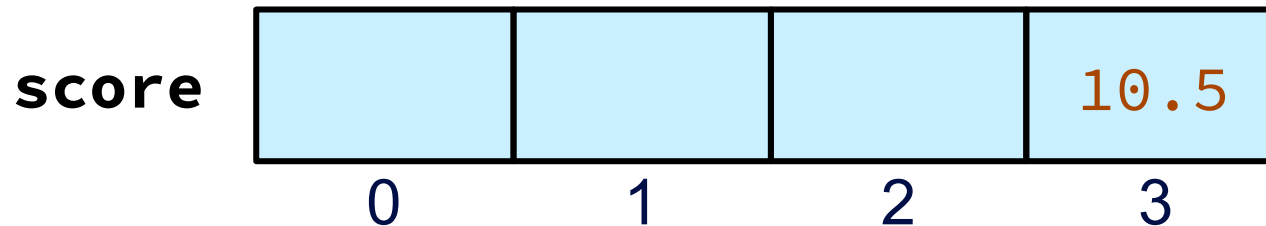


Accessing Arrays

- **Indices** start at **0** in Java.
- **Valid** indices: **0 to size-1**
- Accessing/Assigning Values Examples

```
score[3] = 10.5;
```

```
System.out.println(score[3]);
```



- Two parts of an array:
 - **Fixed part**: array name.
 - **Variable part**: index in square brackets (starts at 0).
 - Example: score[0], score[1], score[2], score[3]

Accessing Arrays

- Java arrays are **zero-indexed**:
 - First index: 0
 - Last index: length - 1
- Trying to access an index **outside the valid range** causes a **runtime error** known as `ArrayIndexOutOfBoundsException`
- What is the **output**?

```
double[] score = new double[4];  
score[4] = 10.0; // IndexOutOfBoundsException
```

score	10.5	17.3	10.5	2.25
	0	1	2	3

Referring to Arrays and Array Elements



- An array **element** is accessed using an indexed expression:
 - Example → `score[2]`
- The **entire array** is referred to using its name without brackets:
 - Example → `score`
 - Typically used for passing arrays as method parameters.

Uses of Square Brackets []

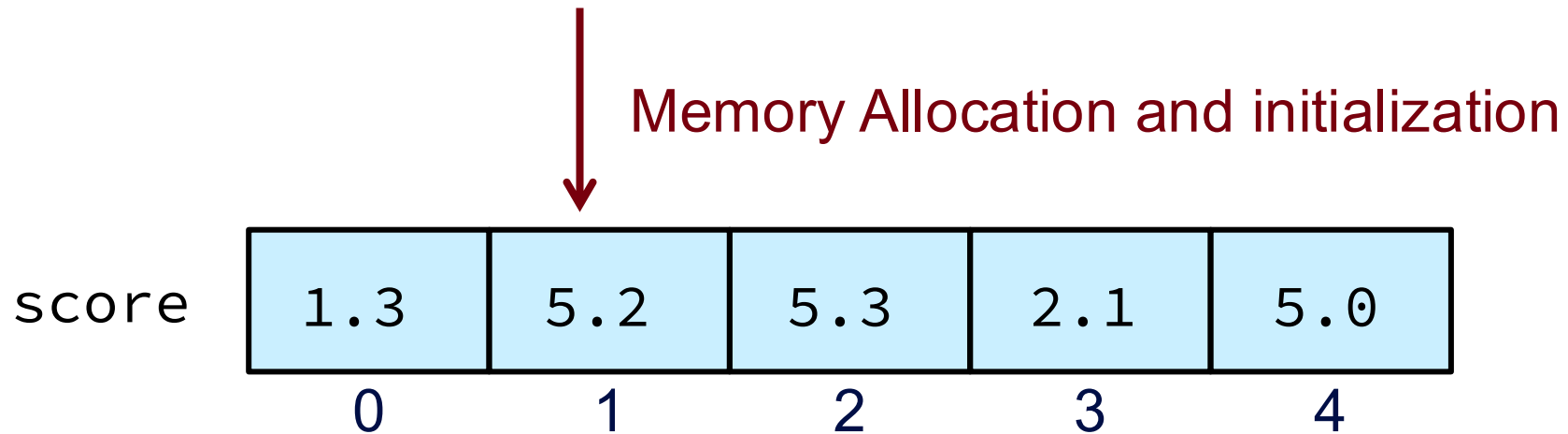


- Accessing an element of an existing array:
 - Example → `max = score[2];`
- Creating a new array with a specific size:
 - Example → `score = new double[5];`
- Declaring an array type (indicating that a variable is an array):
 - Example → `double[] score;`

Initializing Arrays

1. Arrays can be initialized when they are **declared** by using **braces** `{}` with **comma-separated values**.

– Example → `double[] score = {1.3, 5.2, 5.3, 2.1, 5.0}`



- **No** need for `new` when using this form.
- The **size is inferred** from the number of values.

Initializing Arrays



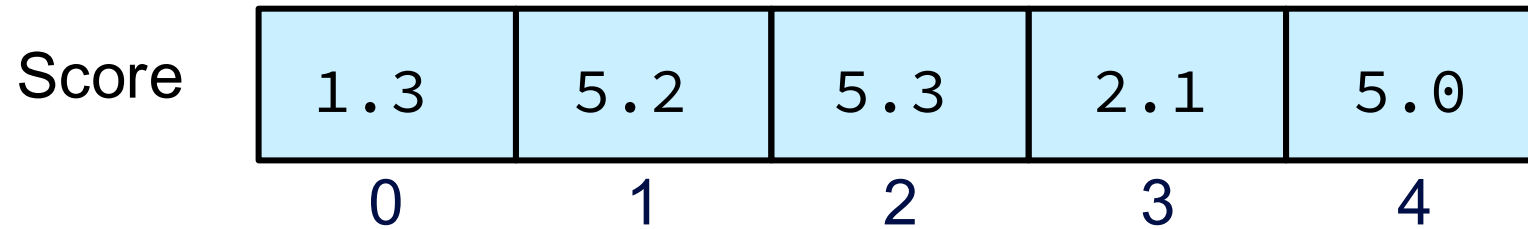
2. Arrays can be initialized **with a for loop**, especially for **larger sizes**.

```
double[] reading = new double[100];  
  
for (int index = 0; index < reading.length; index++)  
    reading[index] = 42.0;
```

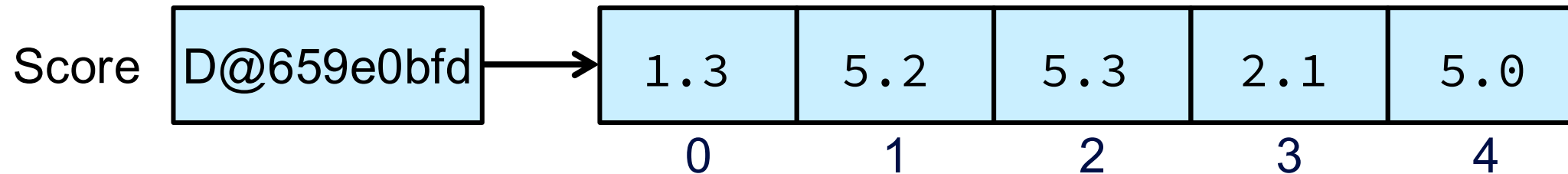
3. If the elements of an array are **not** initialized explicitly, they will automatically be initialized to the **default value** for their base type.

Arrays are Objects

```
double[] score = {1.3, 5.2, 5.3, 2.1, 5.0}
```



is an object

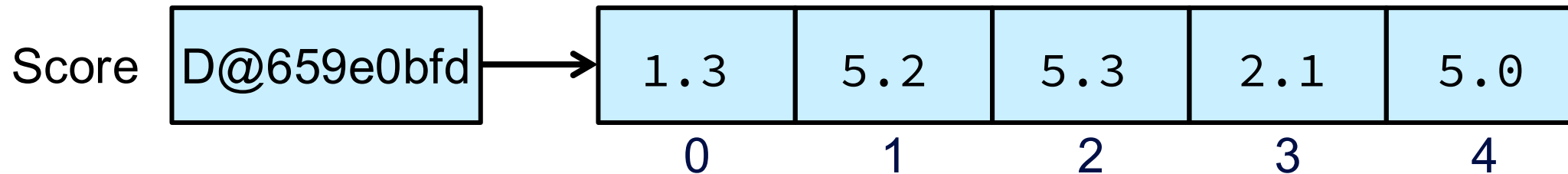


Arrays are Objects



```
double[] score = new double[5];
```

- The variable **score** contains a reference, not the actual data.
- All array variables behave this way — they store memory addresses.
- The **new** expression creates an array that invokes a **constructor** that uses a nonstandard syntax. (There are [] but not () .)



What is the output?



```
int [] values = {1,2,3};  
for (int i = 0; i < values.length; i++)  
    System.out.println(values);
```

The Output will be:

```
[I@659e0bfd]  
[I@659e0bfd]  
[I@659e0bfd]
```

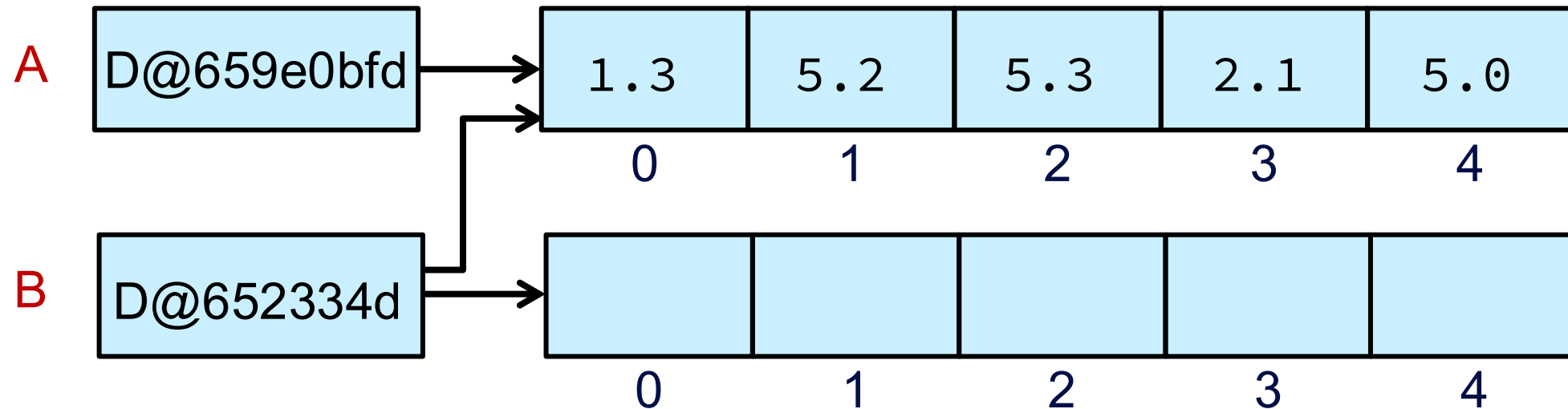
```
int [] values = {1,2,3};  
for (int i = 0; i < values.length; i++)  
    System.out.println(values[i]);
```

The Output will be:

```
1  
2  
3
```

Understanding = and == with arrays

- Arrays are reference types – so their behavior is **similar to objects** in assignment, comparison, and method passing.
- For example:
 - if **A** and **B** are **double** arrays. What will happen if we use **B = A**?



Shallow Copy

Understanding = and == with arrays



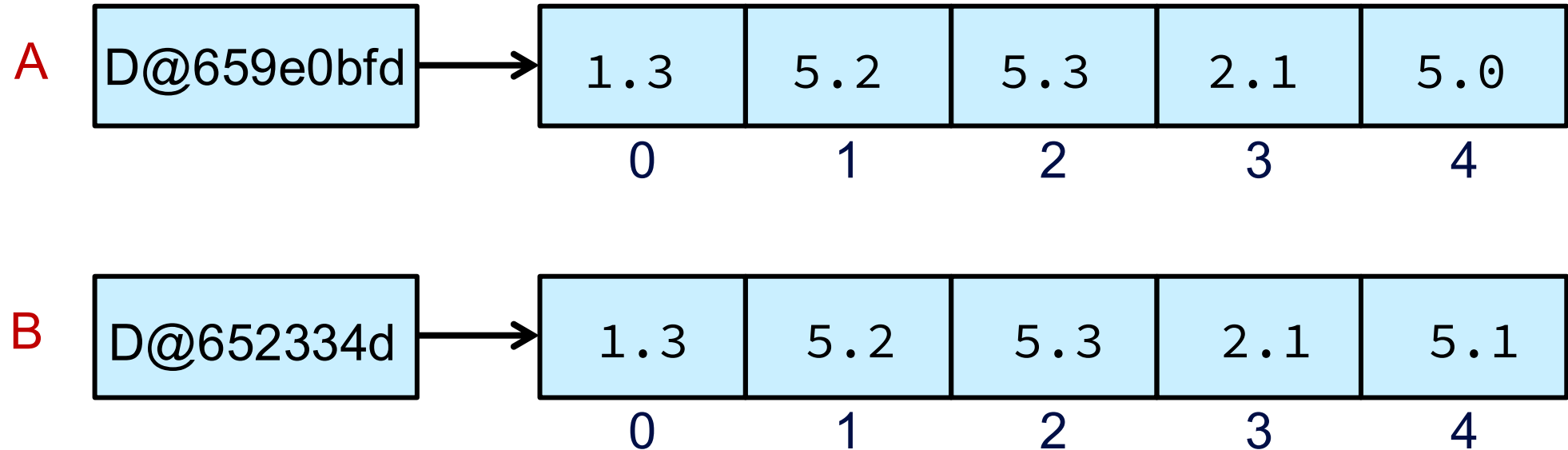
- To make an actual copy of the array **A**'s contents to **B** (a deep copy) we can **use a loop**.

```
for (int i = 0; (i < A.length) && (i < B.length); i++){  
    B [i] = A [i];  
}
```

- This above code does **not make a full copy** unless `A.length == B.length`.

Understanding = and == with arrays

- The `==` operator checks if two arrays refer to the **same** memory location.
- The result of `B == A` will be **true** only if **A** and **B** point to the **same** array



Understanding = and == with arrays

- The == operator checks if two arrays refer to the **same** memory location.
- The result of `B == A` will be **true** only if **A** and **B** point to the **same** array

```
public static boolean equalsArray (int[] a, int[] b) {  
    if (a.length != b.length)  
        return false;  
    else {  
        for (int i = 0; i < a.length; i++ ) {  
            if (a [i] != b [i])  
                return false;  
        }  
    }  
    return true;  
}
```

Methods that return arrays

- A method can **return** an array.
- The **return type** is specified like an **array** parameter.

```
public static int[] doubleValues (int[] input) {  
    int[] result = new int[input.length];  
    for (int i = 0; i < input.length; i++){  
        result [i]= input [i]*2;  
    }  
    return result;  
}  
  
public static void main (String[] args) {  
    int [] original  = {1,2,3};  
    int [] doubleNum  = doubleValues (original);  
}
```

Privacy Leaks with Array Member Variables



- Returning an array from an accessor can **leak private** data.

```
public double[] getArray () {  
    return anArray;  
}
```

- External code can modify the array:

```
var.getArray()[0] = 1; // Directly changes private data!
```

Privacy Leaks with Array Member Variables



- Solution: Return a **Deep Copy**

```
public double [] getArray ()
{
    double [] tmp = new double [anArray.length];
    for (int i = 0; i < anArray.length; i++)
        tmp[i] = anArray[i];
    return tmp;
}
```

Privacy Leaks with Array Member Variables



- Deep Copy for Arrays of Objects
 - If the array holds objects, you must **also copy each object** inside the array to avoid leaks

```
public Car[] getCars() {  
    Car[] copy = new Car[cars.length];  
    for (int i = 0; i < cars.length; i++) {  
        copy[i] = new Car(cars[i]); // Deep copy again for safety  
    }  
    return copy;  
}
```

char[] is **Not** a String



- `char[]` is **conceptually** like a String (sequence of characters) but `char[]` is **not** a String **object**

```
char [] a = {'A', 'B', 'C'};  
String s = a; // Illegal!
```

How to do it? Use a **constructor** to convert:

```
char [] a = {'A', 'B', 'C'};  
String s = new String(a); // s == "ABC"  
String s2 = new String(a, 0, 3); // s2 == "ABC"
```

- **OR** if you want a **subrange** e.g., `String s2 = new String(a, 0, 2);`

This constructor takes →

`String (char[], the starting index, the number of characters to copy from the array)`

char[] is **Not** a String



- An array of characters does have some things in **common** with String objects. For example, unlike other arrays, an array of characters can be output using println

```
char [] a = {'A', 'B', 'C'};  
System.out.println(a);
```

The Output will be:

ABC

The "for-each" Loop



- The standard Java libraries include a number of **collection** classes: classes whose objects store a collection of values.
- Traditional for loops **don't work well** with collections (no indices).
- Java 5 introduced the enhanced for loop (or **for-each**) to simplify looping.

```
for (ArrayBaseType variableName : arrayName ) {  
    Statement;  
}
```

- Reads as: “for each variableName in arrayName, do the following”
- **Works** on arrays and collections
- variableName **must be declared within** the loop
- Uses a colon (:), not a semicolon

The "for-each" Loop

A regular for loop:

```
int [] values = {1,2,3};  
for (int i = 0; i < values.length; i++)  
    System.out.println(values[i]);
```

Foreach loop:

```
int [] values = {1,2,3};  
  
for (int value : values ) {  
    System.out.println(value);  
}
```



THE UNIVERSITY OF
MELBOURNE

Live Coding – Part 2 (array & safe getter)

Live Coding Instruction



- In Student.java,
 - Create an array of two Subjects for a student, remember to initialize it.
 - Allow to set subject ID (in [0, 1]) and name.
 - Get the array of all subjects safely. Can also try an unsafe one to compare the difference.
- In Program.java,
 - Create a Student instance and set two subjects for the student.
 - Print this student's subjects after getting the array of the subject.
 - Change one subject name in the **returned array**.
 - Re-get the student's subject array and print it, see if it is affected by the change above.

See detailed instructions in Ed Workspace/week5.

Additional Reading Resources



- WALTER, S. Absolute Java, Global Edition. [Harlow]: Pearson, 2016. (Chapter 6)
- SCHILDT, H. Java: The Complete Reference, 12th Edition: McGraw-Hill, 2022 (Chapter 3)
- Array (accessible on 14-02-2024) Oracle's Java Documentation. Available at: <https://docs.oracle.com/javase/tutorial/java/nutsandbolts/arrays.html>.
- Arrays (accessible on 14-02-2024) Java 7 Documentation. Available at: <https://docs.oracle.com/javase/specs/jls/se7/html/jls-10.html>.

(c) The University of Melbourne 2023

This material is protected by copyright. Unless indicated otherwise, the University of Melbourne owns the copyright subsisting in the work. Other than for the purposes permitted under the Copyright Act 1968, you are prohibited from downloading, republishing, retransmitting, reproducing or otherwise using any of the materials included in the work as standalone files. Sharing University teaching materials with third-parties, including uploading lecture notes, slides or recordings to websites constitutes academic misconduct and may be subject to penalties. For more information: [Academic Integrity at the University of Melbourne](#)

Requests and enquiries concerning reproduction and rights should be addressed to the University Copyright Office, The University of Melbourne: copyright-office@unimelb.edu.au

The University of Melbourne (Australian University)
PRV12150/CRICOS 00116K

See you next time!



THE UNIVERSITY OF
MELBOURNE